What is JOIN?

In MySQL, **JOIN** is used to **combine data from two or more tables** based on a **common column**.

JOIN is needed because:

- Data is stored in **multiple tables**
- We want to **fetch related data together**

Why JOIN is Used?

- To avoid data duplication
- To maintain relationships between tables
- To fetch meaningful combined results

Example Situation (Without JOIN)

We have **two tables**:

Table 1: students

roll_no	name	city_id
1	Aman	101
2	Riya	102

Table 2: cities

city_id	city_name
101	Lucknow
102	Delhi

If we want **student name + city name together**, we must use **JOIN**.

Types of JOIN in MySQL

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN (not directly supported in MySQL)

INNER JOIN is the most commonly used JOIN, so now we study it in detail.

INNER JOIN in MySQL

INNER JOIN returns **only those records** where the **matching value exists in both tables**.

If data does not match in both tables, it is **not shown**.

INNER JOIN Syntax

```
SELECT column_names
FROM table1
INNER JOIN table2
ON table1.common_column = table2.common_column;
```

INNER JOIN Example (Students & Cities)

```
SELECT students.name, cities.city_name
FROM students
INNER JOIN cities
ON students.city_id = cities.city_id;
```

How INNER JOIN Works (Step-by-Step)

1. MySQL checks `students.city_id`
2. MySQL checks `cities.city_id`
3. Only **matching city_id values** are selected
4. Combined result is displayed

INNER JOIN Output

name	city_name
Aman	Lucknow
Riya	Delhi

Only matching records
Non-matching records are ignored

INNER JOIN with WHERE Clause

```
SELECT students.name, cities.city_name
FROM students
INNER JOIN cities
ON students.city_id = cities.city_id
WHERE cities.city_name = 'Lucknow';
```

Output

name	city_name
Aman	Lucknow

Important Points (Exam Focus)

- INNER JOIN returns **common data**
- Matching column is usually **Primary Key ↔ Foreign Key**

- Non-matching rows are **not included**

JOIN vs INNER JOIN

JOIN and INNER JOIN are the same in MySQL

```
SELECT *
FROM students
JOIN cities
ON students.city_id = cities.city_id;
```

This query works same as INNER JOIN

One-Line Exam Definitions

- **JOIN**: Used to combine rows from two or more tables.
- **INNER JOIN**: Returns only matching records from both tables.

What is LEFT JOIN?

A **LEFT JOIN** returns:

- **All records from the LEFT table**
- **Matching records from the RIGHT table**
- If no match is found, **NULL** is shown for right table columns

LEFT table = table written **before** LEFT JOIN

LEFT JOIN Syntax

```
SELECT column_names
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

LEFT JOIN Example

Table: students (LEFT table)

roll_no	name	city_id
1	Aman	101
2	Riya	102
3	Mohan	105

Table: cities (RIGHT table)

city_id	city_name
101	Lucknow

city_id	city_name
102	Delhi

LEFT JOIN Query

```
SELECT students.name, cities.city_name
FROM students
LEFT JOIN cities
ON students.city_id = cities.city_id;
```

LEFT JOIN Output

name	city_name
Aman	Lucknow
Riya	Delhi
Mohan	NULL

All students are shown
City name shown only if match exists
No city for Mohan → NULL

Key Points of LEFT JOIN

- Left table data is **always shown**
- Right table data appears **only if matched**
- Unmatched right table values → **NULL**

What is RIGHT JOIN?

A **RIGHT JOIN** returns:

- **All records from the RIGHT table**
- **Matching records from the LEFT table**
- If no match is found, **NULL** is shown for left table columns

Right table = table written **after** RIGHT JOIN

RIGHT JOIN Syntax

```
SELECT column_names
FROM table1
RIGHT JOIN table2
ON table1.common_column = table2.common_column;
```

RIGHT JOIN Example

Table: students (LEFT table)

roll_no	name	city_id
1	Aman	101
2	Riya	102

Table: cities (RIGHT table)

city_id	city_name
101	Lucknow
102	Delhi
103	Kanpur

RIGHT JOIN Query

```
SELECT students.name, cities.city_name
FROM students
RIGHT JOIN cities
ON students.city_id = cities.city_id;
```

RIGHT JOIN Output

name	city_name
Aman	Lucknow
Riya	Delhi
NULL	Kanpur

All cities are shown
Student name shown only if match exists
No student in Kanpur → NULL

Key Points of RIGHT JOIN

- Right table data is **always shown**
- Left table data appears **only if matched**
- Unmatched left table values → **NULL**

LEFT JOIN vs RIGHT JOIN (Exam Table)

Feature	LEFT JOIN	RIGHT JOIN
Priority table	Left table	Right table
All rows from	Left table	Right table
Unmatched values	NULL (right)	NULL (left)
Usage	More common	Less common

Easy Trick to Remember

- **LEFT JOIN** → "LEFT table is more important"
- **RIGHT JOIN** → "RIGHT table is more important"

What is CROSS JOIN?

A **CROSS JOIN** returns the **Cartesian Product** of two tables.

This means:

- **Each row of the first table** is combined with
- **Each row of the second table**

No matching condition is required.

Important Points of CROSS JOIN

- It does **not use ON condition**
- Total rows =
  (rows in table1 × rows in table2)
- Can produce a **very large result**
- Used rarely in real projects

CROSS JOIN Syntax

```
SELECT column_names
FROM table1
CROSS JOIN table2;
```

Example Tables

Table: students

roll_no	name
1	Aman
2	Riya

Table: courses

course_id	course_name
C1	MySQL
C2	Python

CROSS JOIN Query

```
SELECT students.name, courses.course_name
FROM students
CROSS JOIN courses;
```

CROSS JOIN Output

name	course_name
Aman	MySQL
Aman	Python
Riya	MySQL
Riya	Python

2 students × 2 courses = **4 rows**

How CROSS JOIN Works (Step-by-Step)

1. Take first row from students
2. Match it with **all rows** from courses
3. Repeat for every student
4. Final result contains **all possible combinations**

CROSS JOIN Without Keyword (Old Style)

```
SELECT students.name, courses.course_name
FROM students, courses;
```

This also creates a CROSS JOIN but **not recommended**

Where is CROSS JOIN Used?

- Generating combinations
- Creating sample data
- Timetable or pairing scenarios
- Testing purposes

CROSS JOIN vs INNER JOIN

Feature	CROSS JOIN	INNER JOIN
Condition	Not required	Required
Matching	All combinations	Only matching
Result size	Very large	Limited
Usage	Rare	Very common

What is Multiple JOIN?

Multiple JOIN means joining **more than two tables** in a single SQL query.

It is used when data is **spread across many related tables**.

Why Multiple JOIN is Used?

- To fetch **complete information** from different tables
- To avoid data duplication
- To show **real-world relationships** (Student → City → State)

Example Scenario

We have **three tables**:

Table 1: students

student_id	name	city_id
1	Aman	101
2	Riya	102

Table 2: cities

city_id	city_name	state_id
101	Lucknow	201
102	Delhi	202

Table 3: states

state_id	state_name
201	Uttar Pradesh
202	Delhi

Multiple JOIN Syntax

```
SELECT column_names
FROM table1
JOIN table2
ON condition1
JOIN table3
ON condition2;
```

Multiple INNER JOIN Example

```
SELECT students.name, cities.city_name, states.state_name
FROM students
INNER JOIN cities
ON students.city_id = cities.city_id
INNER JOIN states
ON cities.state_id = states.state_id;
```

Output

name	city_name	state_name
Aman	Lucknow	Uttar Pradesh
Riya	Delhi	Delhi

Data fetched from **3 tables**
Proper relationships used

How Multiple JOIN Works (Step-by-Step)

1. `students` is joined with `cities`
2. Result is joined with `states`
3. Matching rows are returned
4. Final combined data is shown

Multiple JOIN with LEFT JOIN Example

```
SELECT students.name, cities.city_name, states.state_name
FROM students
LEFT JOIN cities
ON students.city_id = cities.city_id
LEFT JOIN states
ON cities.state_id = states.state_id;
```

All students will be shown, even if city or state is missing.

Important Points (Exam Focus)

- Multiple JOIN works **step by step**
- Order of JOIN matters
- PK–FK relationship is usually used
- Any JOIN type can be used (INNER, LEFT, RIGHT)

Real-Life Example

- Student → Course → Teacher
- Order → Customer → Payment
- Employee → Department → Company

Multiple JOIN vs Single JOIN

Feature	Single JOIN	Multiple JOIN
Tables joined	2	More than 2
Complexity	Simple	Moderate
Usage	Basic queries	Real projects

What is a Subquery?

A **Subquery** is a **query inside another query**.

The inner query runs **first**, and its result is used by the outer query.

Where Subqueries Can Be Used

- In `SELECT` clause
- In `WHERE` clause
- In `FROM` clause

Basic Subquery Syntax

```
SELECT column_name
FROM table
WHERE column_name = (SELECT column_name FROM table);
```

Example Tables

students

student_id	name	city_id
1	Aman	101
2	Riya	102
3	Mohan	103

cities

city_id	city_name
101	Lucknow
102	Delhi

Subquery Example (Single Row Result)

Find students who live in Delhi

```
SELECT name
FROM students
WHERE city_id = (
    SELECT city_id
    FROM cities
    WHERE city_name = 'Delhi'
);
```

Inner query runs first
Outer query uses its result

Subquery with IN (Multiple Rows)

```
SELECT name
FROM students
WHERE city_id IN (
    SELECT city_id
    FROM cities
);
```

Returns students whose city exists in cities table

What is EXISTS?

EXISTS checks **whether a subquery returns any row or not**.

It returns **TRUE or FALSE**
It does **not return data**, only checks existence

EXISTS Syntax

```
SELECT column_name
FROM table1
WHERE EXISTS (
    SELECT column_name
    FROM table2
    WHERE condition
);
```

EXISTS Example

Find students whose city exists in cities table

```
SELECT name
FROM students s
WHERE EXISTS (
    SELECT 1
    FROM cities c
    WHERE s.city_id = c.city_id
);
```

How EXISTS Works

- Subquery checks row by row
- If **any match is found**, result is TRUE
- Query stops checking further rows

Very efficient for large data

What is NOT EXISTS?

NOT EXISTS returns rows **when the subquery returns no result**.

It is the opposite of EXISTS

NOT EXISTS Syntax

```
SELECT column_name
FROM table1
WHERE NOT EXISTS (
    SELECT column_name
    FROM table2
    WHERE condition
);
```

NOT EXISTS Example

Find students whose city does NOT exist in cities table

```
SELECT name
FROM students s
WHERE NOT EXISTS (
    SELECT 1
    FROM cities c
    WHERE s.city_id = c.city_id
);
```

Output

name
Mohan

Mohan's city_id (103) does not exist in cities table

EXISTS vs IN (Important for Exams)

Feature	EXISTS	IN
Performance	Faster	Slower (large data)
NULL handling	Safe	Can cause issues
Result	TRUE / FALSE	List of values
Usage	Correlated queries	Simple checks

What is GROUP BY?

GROUP BY is used to **group rows that have the same values** in one or more columns.

It is mainly used with **aggregate functions** like:

- `COUNT()`
- `SUM()`
- `AVG()`
- `MAX()`
- `MIN()`

Why GROUP BY is Used?

- To summarize data
- To calculate totals, averages, counts for each group
- To generate reports

GROUP BY Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name;
```

Example Table: students

student_id	name	city	marks
1	Aman	Delhi	80
2	Riya	Delhi	90
3	Mohan	Lucknow	70
4	Neha	Lucknow	85

GROUP BY Example

Count students in each city

```
SELECT city, COUNT(*) AS total_students
FROM students
GROUP BY city;
```

Output

city	total_students
Delhi	2
Lucknow	2

GROUP BY with SUM

```
SELECT city, SUM(marks) AS total_marks
FROM students
GROUP BY city;
```

Important Rules of GROUP BY

- Columns in `SELECT` must be:
  - In `GROUP BY`, **or**
  - Used with aggregate functions
- GROUP BY works **after WHERE**

What is HAVING?

HAVING is used to **filter grouped data**.

It is used with **GROUP BY**
WHERE cannot be used with aggregate results

Difference Between WHERE and HAVING

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions	Can use aggregate functions

HAVING Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

HAVING Example

Show only cities with more than 1 student

```
SELECT city, COUNT(*) AS total_students
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

Output

city	total_students
Delhi	2
Lucknow	2

GROUP BY with WHERE and HAVING (Combined)

```
SELECT city, AVG(marks) AS avg_marks
FROM students
WHERE marks > 75
GROUP BY city
HAVING AVG(marks) > 80;
```

Order of Execution (Very Important)

1. FROM
2. WHERE

3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

- **HAVING**: Filters grouped results after GROUP BY.

Easy Trick to Remember

- **WHERE** → filter rows
- **GROUP BY** → make groups
- **HAVING** → filter groups

What is a Database Transaction?

A **Transaction** is a **set of SQL operations** that are executed as **one single unit of work**.

Either **all operations succeed** or **none of them are applied**.

Why Transactions are Needed?

- To maintain **data consistency**
- To avoid **partial updates**
- To handle **errors safely**
- Very important in **banking, payments, orders**

Real-Life Example

Bank Transfer:

- Debit money from Account A
- Credit money to Account B

If debit succeeds but credit fails → **rollback needed**

Transaction Control Statements (TCL)

Command	Purpose
START TRANSACTION / BEGIN	Start a transaction
COMMIT	Save changes permanently
ROLLBACK	Undo changes
SAVEPOINT	Set a rollback point

Basic Transaction Syntax

```
START TRANSACTION;

-- SQL statements
```

```
COMMIT;
```

Example Table: accounts

acc_id	name	balance
1	Aman	5000
2	Riya	3000

Transaction Example (Money Transfer)

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE acc_id = 1;

UPDATE accounts
SET balance = balance + 1000
WHERE acc_id = 2;

COMMIT;
```

Both updates succeed → changes saved

ROLLBACK Example

```
START TRANSACTION;

UPDATE accounts
SET balance = balance - 1000
WHERE acc_id = 1;

-- Suppose error occurs here

ROLLBACK;
```

All changes are undone

What is SAVEPOINT?

A **SAVEPOINT** allows you to **rollback to a specific point** in a transaction.

SAVEPOINT Example

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE acc_id = 1;
SAVEPOINT sp1;

UPDATE accounts SET balance = balance - 500 WHERE acc_id = 1;

ROLLBACK TO sp1;
```

```
COMMIT;
```

Only second update is undone
First update remains

What is AUTOCOMMIT?

- By default, MySQL runs in **AUTOCOMMIT mode**
- Every statement is committed automatically

Disable AUTOCOMMIT

```
SET autocommit = 0;
```

Enable AUTOCOMMIT

```
SET autocommit = 1;
```

ACID Properties of Transaction (Very Important)

Property	Meaning
Atomicity	All or nothing
Consistency	Data remains valid
Isolation	Transactions don't interfere
Durability	Data saved permanently

Storage Engine Note (Exam Point)

Transactions work **only with InnoDB** engine
Not supported in MyISAM

Transaction Flow (Easy to Remember)

```
START → SQL → COMMIT
         ↓
      ROLLBACK
```

One-Line Definitions

- **Transaction**: A group of SQL statements executed as a single unit.
- **COMMIT**: Saves transaction changes permanently.
- **ROLLBACK**: Cancels transaction changes.
- **SAVEPOINT**: Marks a point inside a transaction.